Modul 7 — JavaScript Dasar sampai Menengah: Fondasi Logika dan Interaktivitas Web

Mata pelajaran: Pemrograman Web — RPL
Alat: VS Code, browser, DevTools (`F12`), dan browser console
Studi kasus: Aplikasi data siswa berbasis web
Prasyarat: HTML dasar, CSS dasar, logika pemrograman dasar, dan pemahaman tentang struktur data sederhana

---

1. Tujuan Pembelajaran

Setelah mempelajari modul ini, kamu mampu:

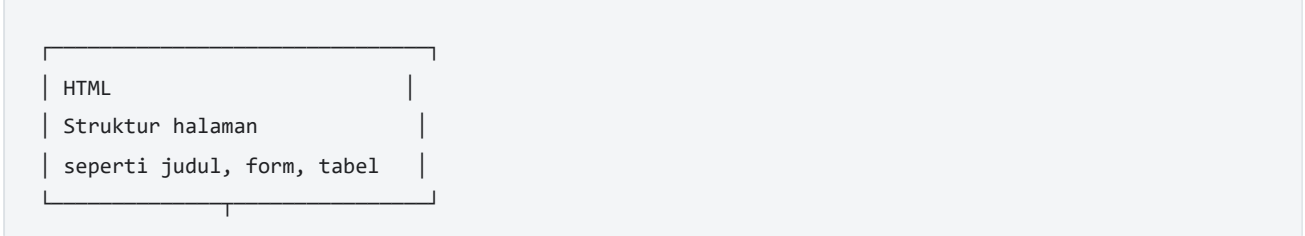
1. Menjelaskan fungsi JavaScript di browser dan hubungannya dengan HTML dan CSS.
2. Menggunakan variabel, tipe data, operator, kondisional, dan perulangan dengan benar.
3. Menulis fungsi JavaScript yang reusable dan mudah dibaca.
4. Mengolah data array dan object untuk kebutuhan aplikasi.
5. Memahami konsep `sync` dan `async` serta bagaimana JavaScript bekerja dalam eksekusi program.
6. Menggunakan DOM untuk membaca, menambah, dan mengubah elemen halaman.
7. Menangani event pengguna seperti `click` , `submit` , dan `input` .
8. Membuat validasi form, pencarian data, dan penyimpanan preferensi dengan `localStorage` .
9. Menggunakan `fetch` , `Promise` , dan `async/await` untuk mengambil data dari server.

---

2. Konsep Dasar

2.1 JavaScript bekerja di browser

JavaScript adalah bahasa pemrograman yang biasanya berjalan di browser untuk membuat halaman web menjadi lebih interaktif.





Tanpa JavaScript, halaman web cenderung statis. Dengan JavaScript, halaman bisa berubah saat user melakukan tindakan seperti klik, mengetik, atau mengirim form.

2.2 Browser vs server

Komponen	Tugas utama	Contoh
HTML	Menyusun struktur halaman	form, tabel, tombol
CSS	Mengatur tampilan	warna, font, grid
JavaScript	Mengelola logika dan interaksi	validasi, pencarian, alert
PHP	Memproses data di server	query database, login
MySQL	Menyimpan data	tabel siswa, jurusan, nilai

> JavaScript biasanya bekerja di sisi klien (browser), sedangkan PHP dan MySQL bekerja di sisi server. Keduanya saling melengkapi.

2.3 Mengapa JavaScript penting?

JavaScript dipakai untuk:

- validasi form sebelum dikirim
- membuka dan menutup menu
- menampilkan pesan sukses atau error
- membuat pencarian langsung (live search)
- menampilkan data dari server tanpa reload
- menyimpan pengaturan user seperti tema gelap

---

3. JavaScript Dasar: Dasar-dasar Logika

3.1 Menulis JavaScript pertama

```
<script>
  console.log("Halo, dunia!");
</script>
```

Atau file terpisah:

```
console.log("Halo, dunia!");
```

Buka browser lalu tekan `F12` → tab **Console**. Nanti hasilnya akan muncul di sana.

3.2 Variabel

Variabel adalah wadah untuk menyimpan data.

```
let nama = "Andi";
const umur = 17;
var kelas = "XI RPL";
```

Penjelasan:

- `let` → variabel yang bisa diubah nilainya.
- `const` → variabel yang nilainya tetap.
- `var` → cara lama, sebaiknya tidak dipakai lagi untuk kode modern.

Contoh:

```
let nilai = 80;
nilai = 90;
console.log(nilai); // 90
```

3.3 Tipe data

JavaScript memiliki beberapa tipe data utama:

```
let angka = 25;                 // number
let nama = "Andi";             // string
let aktif = true;               // boolean
let kosong = null;              // null
let tidakAda;                   // undefined
let siswa = ["Andi", "Budi"]; // array
let profil = { nama: "Siti" }; // object
```

3.4 Operator

Operator digunakan untuk menghitung atau membandingkan data.

```
let a = 10;
let b = 5;

console.log(a + b); // 15
console.log(a - b); // 5
console.log(a * b); // 50
console.log(a / b); // 2
console.log(a % b); // 0
console.log(a > b); // true
console.log(a === b); // false
```

3.5 Kondisi (`if`, `else`, `switch`)

`if` dan `else`

```
let nilai = 80;

if (nilai >= 75) {
  console.log("Lulus");
} else {
  console.log("Tidak lulus");
}
```

`switch`

```
let hari = 2;

switch (hari) {
  case 1:
    console.log("Senin");
    break;
  case 2:
    console.log("Selasa");
    break;
  default:
    console.log("Hari lain");
}
```

3.6 Perulangan

`for`

```
for (let i = 1; i <= 5; i++) {
  console.log(i);
}
```

`while`

```
let i = 0;
while (i < 3) {
  console.log(i);
  i++;
}
```

`for...of`

```
const siswa = ["Andi", "Budi", "Rina"];

for (const nama of siswa) {
  console.log(nama);
}
```

`for...in`

```
const profil = { nama: "Dewi", kelas: "XI RPL" };

for (const key in profil) {
  console.log(key + ": " + profil[key]);
}
```

> Perulangan sangat penting dalam aplikasi data siswa karena kita sering menampilkan daftar siswa satu per satu dari array atau hasil query database.

---

4. Fungsi dalam JavaScript

Fungsi adalah blok kode yang dapat dipanggil berulang kali. Fungsi membuat program lebih rapi dan reusable.

4.1 Fungsi dengan parameter

```
function sapa(nama) {
  return `Halo, ${nama}!`;
}

console.log(sapa("Rina"));
console.log(sapa("Andi"));
```

4.2 Fungsi tanpa parameter

```
function tampilJudul() {
  console.log("Daftar Siswa");
}

tampilJudul();
```

4.3 Function declaration dan function expression

Function declaration

```
function tambah(a, b) {
  return a + b;
}

console.log(tambah(10, 5));
```

Function expression

```
const kali = function (a, b) {
  return a * b;
};

console.log(kali(4, 3));
```

4.4 Arrow function

Arrow function adalah bentuk singkat dari fungsi.

```
const luasPersegi = (sisi) => sisi * sisi;
console.log(luasPersegi(8));
```

Arrow function sering dipakai dalam callback, seperti saat looping array.

4.5 Callback

Callback adalah fungsi yang dipanggil di dalam fungsi lain.

```
const siswa = ["Andi", "Budi", "Siti"];

siswa.forEach((nama) => {
  console.log(`Siswa: ${nama}`);
});
```

4.6 Fungsi dengan nilai default

```
function hitungNilai(nilai, bonus = 0) {
  return nilai + bonus;
}

console.log(hitungNilai(80));
console.log(hitungNilai(80, 10));
```

4.7 Mengapa fungsi itu penting?

Dalam aplikasi, fungsi dipakai untuk:

- validasi input
- menghitung rata-rata nilai
- mencari siswa berdasarkan nama
- mengubah tampilan halaman
- memproses data API

---

5. Array dan Object

5.1 Array

Array adalah kumpulan data yang disimpan dalam satu variabel.

```
const nilai = [80, 90, 75, 85];

console.log(nilai[0]);
console.log(nilai.length);
console.log(nilai[2]);
```

Array method yang sering dipakai

```
const nilai = [80, 90, 75, 85];

console.log(nilai.map((n) => n + 5));
console.log(nilai.filter((n) => n >= 80));
console.log(nilai.reduce((total, n) => total + n, 0));
```

`map` → membuat array baru
`filter` → menyaring data
`reduce` → menjumlahkan atau menggabungkan data

5.2 Object

Object adalah kumpulan data dalam bentuk pasangan `key` dan `value`.

```
const siswa = {
  nama: "Dewi",
  kelas: "XI RPL",
  nilai: 90,
};

console.log(siswa.nama);
console.log(siswa["kelas"]);
```

5.3 Array of object

```
const dataSiswa = [
  { nama: "Andi", nilai: 80 },
  { nama: "Siti", nilai: 90 },
  { nama: "Budi", nilai: 75 },
];

console.log(dataSiswa[0].nama); // Andi
console.log(dataSiswa[1].nilai); // 90
```

5.4 Menghitung rata-rata nilai siswa

```
const dataSiswa = [
  { nama: "Andi", nilai: 80 },
  { nama: "Siti", nilai: 90 },
  { nama: "Budi", nilai: 75 },
  { nama: "Rina", nilai: 95 },
];

const total = dataSiswa.reduce((sum, item) => sum + item.nilai, 0);
const rataRata = total / dataSiswa.length;

console.log(total);      // 340
console.log(rataRata);   // 85
```

5.5 Destructuring

Destructuring membuat kode lebih ringkas saat mengambil data dari array atau object.

```
const siswa = { nama: "Andi", kelas: "XI RPL" };
const { nama, kelas } = siswa;

console.log(nama); // Andi
console.log(kelas); // XI RPL
```

```
const nilai = [80, 90, 75];
const [nilaiPertama, nilaiKedua] = nilai;

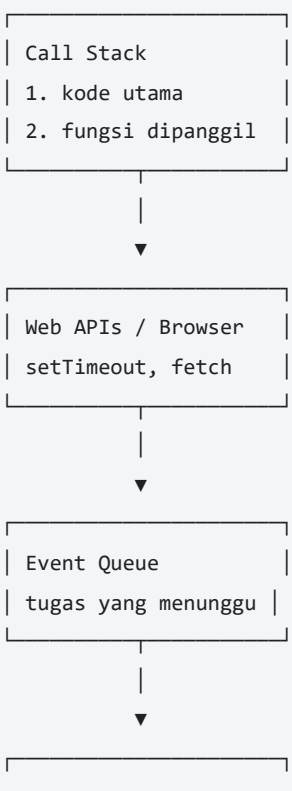
console.log(nilaiPertama); // 80
console.log(nilaiKedua); // 90
```

---

6. JavaScript dan Runtime Browser

6.1 Proses eksekusi JavaScript

JavaScript berjalan dalam satu thread utama browser, tetapi dapat menangani tugas async melalui event loop.



```
| Event Loop           |
| menjalankan tugas    |
└──────────────────────┘
```

6.2 Sync vs Async

Synchronous

```
console.log("A");
console.log("B");
console.log("C");
```

Hasilnya:

```
A
B
C
```

Asynchronous

```
console.log("Mulai");

setTimeout(() => {
  console.log("Tunggu 1 detik");
}, 1000);

console.log("Selesai");
```

Hasilnya:

```
Mulai
Selesai
Tunggu 1 detik
```

Penjelasan:

- `console.log("Mulai")` langsung dijalankan.
- `setTimeout` ditunda ke browser.
- kode lain tetap berjalan.
- setelah 1 detik, callback baru dieksekusi.

6.3 Kenapa async penting?

Dalam aplikasi web, async dipakai untuk:

- membaca data dari server
- menunggu input dari user
- memuat data tanpa reload
- menampilkan pesan saat proses loading selesai

---

7. DOM (Document Object Model)

DOM adalah representasi struktur halaman HTML yang bisa diakses dan diubah oleh JavaScript.

7.1 Pohon DOM sederhana

```
document
└── html
    ├── head
    └── body
        ├── h1
        ├── form
        │   ├── input
        │   └── button
        └── table
            └── tbody
```

7.2 Memilih elemen

```
document.getElementById("judul");
document.querySelector(".card");
document.querySelectorAll("li");
```

Contoh HTML

```
<h1 id="judul">Daftar Siswa</h1>
<p class="pesan">Pesan berhasil</p>
```

Contoh JavaScript

```
const judul = document.getElementById("judul");
const pesan = document.querySelector(".pesan");

console.log(judul.textContent);
console.log(pesan.textContent);
```

7.3 Mengubah isi elemen

```
const judul = document.getElementById("judul");
judul.textContent = "Daftar Siswa Baru";
```

7.4 Mengubah class dan atribut

```
const card = document.querySelector(".card");
card.classList.add("active");
card.classList.remove("hidden");
```

```
const input = document.querySelector("#nama");
input.setAttribute("placeholder", "Masukkan nama siswa");
```

7.5 Membuat elemen baru

```
const item = document.createElement("li");
item.textContent = "Andi";

document.querySelector("ul").appendChild(item);
```

7.6 Menghapus elemen

```
const elemen = document.querySelector(".pesan");
elemen.remove();
```

> Peringatan: jangan memakai `innerHTML` untuk menampilkan data dari pengguna jika datanya tidak aman. Lebih aman memakai `textContent` .

---

8. Event pada JavaScript

Event adalah kejadian yang terjadi pada halaman, seperti klik tombol, menekan keyboard, atau mengirim form.

8.1 Event `click`

```
<button id="tombol">Klik Saya</button>
```

```
document.getElementById("tombol").addEventListener("click", () => {
  alert("Tombol diklik!");
});
```

8.2 Event `submit`

```
<form id="formSiswa">
  <input type="text" id="nama" />
  <button type="submit">Simpan</button>
</form>
```

```
const form = document.getElementById("formSiswa");
form.addEventListener("submit", (event) => {
  event.preventDefault();

  const nama = document.getElementById("nama").value;
  console.log("Nama siswa:", nama);
});
```

8.3 Event `input`

```
const input = document.getElementById("nama");

input.addEventListener("input", (event) => {
  console.log(event.target.value);
});
```

8.4 Event delegation

Event delegation adalah teknik menangani banyak elemen dengan satu listener di parent.

```
<ul id="daftar-siswa">
  <li data-nama="Andi">Andi</li>
  <li data-nama="Budi">Budi</li>
</ul>
```

```
document.getElementById("daftar-siswa").addEventListener("click", (event) => {
  const item = event.target.closest("li");
  if (!item) return;

  console.log("Klik siswa:", item.dataset.nama);
});
```

---

9. Validasi Form

Validasi form membantu user memahami input yang salah sebelum data dikirim ke server.

9.1 Form sederhana

```
<form id="formTambahSiswa">
  <input type="text" id="namaSiswa" placeholder="Nama siswa" />
  <button type="submit">Tambah</button>
</form>
```

```
const formTambah = document.getElementById("formTambahSiswa");

formTambah.addEventListener("submit", (event) => {
  event.preventDefault();

  const nama = document.getElementById("namaSiswa").value.trim();

  if (!nama) {
    alert("Nama siswa tidak boleh kosong");
    return;
  }

  if (nama.length < 3) {
    alert("Nama siswa minimal 3 karakter");
    return;
  }
```

```
  alert(`Data ${nama} berhasil ditambahkan`);
});
```

9.2 Validasi dengan regex

```
const nama = "Andi";
const valid = /^[A-Za-z\s]+$/.test(nama);

console.log(valid); // true
```

9.3 Counter karakter

```
<textarea id="catatan" maxlength="200"></textarea>
<p id="counter">0 / 200</p>
```

```
const catatan = document.getElementById("catatan");
const counter = document.getElementById("counter");

catatan.addEventListener("input", () => {
  const panjang = catatan.value.length;
  counter.textContent = `${panjang} / 200`;
});
```

> Validasi JavaScript membantu user, tetapi tetap harus ada validasi di PHP/MySQL untuk keamanan.

---

10. `localStorage`, `sessionStorage`, dan JSON

10.1 `localStorage`

`localStorage` digunakan untuk menyimpan data di browser. Data ini tetap ada sampai user menghapusnya.

```
localStorage.setItem("tema", "dark");
const tema = localStorage.getItem("tema");
console.log(tema); // dark
```

10.2 `sessionStorage`

`sessionStorage` hanya bertahan selama tab/browser session aktif.

```
sessionStorage.setItem("user", "andi");
console.log(sessionStorage.getItem("user"));
```

10.3 JSON

JSON digunakan untuk mengirim data dalam format teks.

```
const siswa = {
  nama: "Andi",
  kelas: "XI RPL",
  jurusan: "RPL",
};

const teks = JSON.stringify(siswa);
console.log(teks);

const kembali = JSON.parse(teks);
console.log(kembali.nama);
```

10.4 Contoh penggunaan tema gelap

```
const tombolTema = document.getElementById("tombolTema");

tombolTema.addEventListener("click", () => {
  const root = document.documentElement;
  const currentTheme = root.getAttribute("data-theme");

  if (currentTheme === "dark") {
    root.setAttribute("data-theme", "light");
    localStorage.setItem("tema", "light");
  } else {
    root.setAttribute("data-theme", "dark");
    localStorage.setItem("tema", "dark");
  }
});
```

---

11. `fetch`, `Promise`, dan `async/await`

11.1 `fetch` untuk mengambil data

```
fetch("https://jsonplaceholder.typicode.com/users")
  .then((response) => response.json())
  .then((data) => console.log(data));
```

11.2 `Promise`

Promise adalah "janji" bahwa hasil akan tersedia di masa depan.

```
const ambilData = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Data berhasil diambil");
  }, 1000);
});

ambilData.then((hasil) => {
  console.log(hasil);
});
```

11.3 `async/await`

`async/await` membuat kode lebih mudah dibaca dibanding `.then()`.

```
async function ambilSiswa() {
  const response = await fetch("https://jsonplaceholder.typicode.com/users");
  const data = await response.json();
  console.log(data);
}

ambilSiswa();
```

11.4 `try/catch`

```
async function ambilDataSiswa() {
  try {
    const response = await fetch("api_siswa.php");

    if (!response.ok) {
      throw new Error("Gagal mengambil data");
    }

    const data = await response.json();
    console.log(data);
  } catch (error) {
    console.error(error.message);
  }
}
```

11.5 Contoh fetch dengan query pencarian

```
async function cariSiswa(keyword) {
  const response = await fetch(`api_siswa.php?q=${keyword}`);
  const result = await response.json();
  return result;
}

const data = await cariSiswa("andi");
console.log(data);
```

> `fetch` biasanya dipakai untuk ambil data tanpa reload halaman. Ini adalah salah satu fitur paling penting dalam aplikasi web modern.

---

12. Studi Kasus: Aplikasi Data Siswa

12.1 Situasi

Seorang guru ingin melihat daftar siswa, mencari siswa berdasarkan nama, dan mengubah tema aplikasi.

12.2 Fitur yang akan dibuat

No	Fitur	Teknik JavaScript
1	Tombol tema gelap/terang	`localStorage`, `classList`
2	Pencarian siswa real-time	`input`, `fetch`, `async/await`
3	Validasi form siswa	`submit`, `preventDefault`, `if`
4	Tabel hasil data	`DOM`, `createElement`, `appendChild`
5	Pesan sukses	`textContent`, `setTimeout`
6	Sorting sederhana	`sort`, `map`, `reduce`

12.3 Alur kerja aplikasi

```
Halaman terbuka
   │
   ▼
JS mulai dijalankan
   │
   ├── membaca tema dari localStorage
   ├── mengisi data awal ke tabel
   ├── menyiapkan event listener untuk form/cari
   ├── menerima input user
   ├── validasi data
   ├── ambil data dari server atau array lokal
   └── tampilkan hasil ke halaman
```

12.4 Contoh live search sederhana

```
<input id="q" type="text" placeholder="Cari siswa" />
<ul id="hasilCari"></ul>
```

```
const inputCari = document.getElementById("q");
const hasilCari = document.getElementById("hasilCari");

const dataSiswa = ["Andi", "Budi", "Siti", "Rina", "Dewi"];

inputCari.addEventListener("input", () => {
  const kata = inputCari.value.trim().toLowerCase();
  hasilCari.innerHTML = "";

  const hasil = dataSiswa.filter((nama) => nama.toLowerCase().includes(kata));

  hasil.forEach((nama) => {
    const item = document.createElement("li");
    item.textContent = nama;
    hasilCari.appendChild(item);
  });
});
```

12.5 Contoh validasi data siswa

```
const form = document.getElementById("formSiswa");

form.addEventListener("submit", (event) => {
  event.preventDefault();

  const nama = document.getElementById("nama").value.trim();
  const jurusan = document.getElementById("jurusan").value.trim();

  if (!nama || !jurusan) {
    alert("Nama dan jurusan harus diisi");
    return;
  }

  console.log("Data valid:", { nama, jurusan });
});
```

---

13. Array Method yang Sering Dipakai

13.1 `map`

```
const siswa = ["Andi", "Budi", "Siti"];
const siswaUpper = siswa.map((nama) => nama.toUpperCase());
console.log(siswaUpper);
```

13.2 `filter`

```
const nilai = [70, 80, 90, 60];
const lulus = nilai.filter((n) => n >= 75);
console.log(lulus);
```

13.3 `reduce`

```
const nilai = [80, 90, 85];
const total = nilai.reduce((sum, n) => sum + n, 0);
console.log(total);
```

13.4 `sort`

```
const nilai = [90, 70, 80];
const urut = [...nilai].sort((a, b) => a - b);
console.log(urut);
```

13.5 Kombinasi dari banyak method

```
const dataSiswa = [
  { nama: "Andi", nilai: 80 },
  { nama: "Siti", nilai: 95 },
  { nama: "Budi", nilai: 85 },
];

const totalNilai = dataSiswa
  .map((siswa) => siswa.nilai)
  .reduce((sum, nilai) => sum + nilai, 0);

console.log(totalNilai);
```

14. Kesalahan Umum yang Sering Terjadi

14.1 Lupa menulis `;`

```
const nama = "Andi"
console.log(nama)
```

JavaScript tetap bisa berjalan, tetapi kadang menimbulkan bug yang sulit dideteksi.

14.2 Mengubah `const` dengan nilai baru

```
const siswa = "Andi";
// siswa = "Budi"; // ERROR
```

14.3 Mengakses DOM sebelum elemen ada

```
const form = document.getElementById("formSiswa");
form.addEventListener("submit", () => {});
```

Jika script dimuat di atas elemen HTML, `form` bisa bernilai `null`.

Solusinya:

- letakkan script di bawah HTML, atau
- gunakan `DOMContentLoaded`.

```
document.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("formSiswa");
  form.addEventListener("submit", () => {
    console.log("Form siap");
  });
});
```

15. Mini Project: Form Siswa dengan JavaScript

Berikut contoh struktur sederhana:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Daftar Siswa</title>
  </head>
  <body>
    <h1>Daftar Siswa</h1>

    <form id="formSiswa">
      <input type="text" id="nama" placeholder="Nama siswa" />
      <button type="submit">Tambah</button>
    </form>

    <ul id="daftarSiswa"></ul>

    <script>
      const form = document.getElementById("formSiswa");
      const daftarSiswa = document.getElementById("daftarSiswa");

      form.addEventListener("submit", (event) => {
        event.preventDefault();

        const nama = document.getElementById("nama").value.trim();

        if (!nama) {
          alert("Nama tidak boleh kosong");
          return;
        }

        const item = document.createElement("li");
        item.textContent = nama;
        daftarSiswa.appendChild(item);

        form.reset();
      });
    </script>
  </body>
</html>
```

Kode di atas menunjukan bagaimana JavaScript memproses input user, memvalidasi, lalu menampilkan hasil ke browser tanpa reload halaman.

---

16. Kesimpulan

JavaScript dasar sampai menengah adalah fondasi penting dalam membangun aplikasi web yang interaktif.

Dengan memahami:

- variabel, tipe data, perulangan, percabangan,
- fungsi dan callback,
- array dan object,
- DOM dan event,
- validasi form,
- `localStorage`,
- serta `async/await` dan `fetch`,

kamu sudah siap untuk membangun fitur seperti:

- form siswa,
- pencarian data tanpa reload,
- validasi input langsung di browser,
- tema gelap/terang,
- menampilkan daftar data dinamis.

JavaScript bukan hanya untuk tampilan, tetapi juga untuk logika transaksi user di browser — dan ini menjadi fondasi penting sebelum ke materi PHP, database, dan autentikasi.

---

17. Latihan

1. Buat fungsi `hitungRataRata(nilai)` untuk mengolah array nilai siswa.
2. Tampilkan daftar siswa dari array ke dalam tabel HTML.
3. Buat form tambah siswa dengan validasi nama minimal 3 karakter.
4. Buat fitur pencarian siswa berdasarkan input user.
5. Simpan preferensi tema gelap ke `localStorage`.
6. Gunakan `fetch` untuk mengambil data dari file JSON atau API sederhana.
7. Buat tombol untuk menambah dan menghapus item dari daftar siswa.

---

18. Referensi dan Sumber Belajar

- MDN Web Docs — JavaScript
- W3Schools — JavaScript Tutorial
- Developer Mozilla — DOM, Event, Fetch

- Dokumentasi resmi browser Chrome/Edge

---

19. Ringkasan Singkat

```
JavaScript dasar = logika pemrograman di browser
JavaScript menengah = interaksi, validasi, data, dan async
Tujuan utama = membuat website lebih hidup, responsif, dan interaktif
```

Pada tahap berikutnya, materi ini akan menjadi bekal saat kamu masuk ke PHP, MySQL, dan sistem autentikasi mini project.